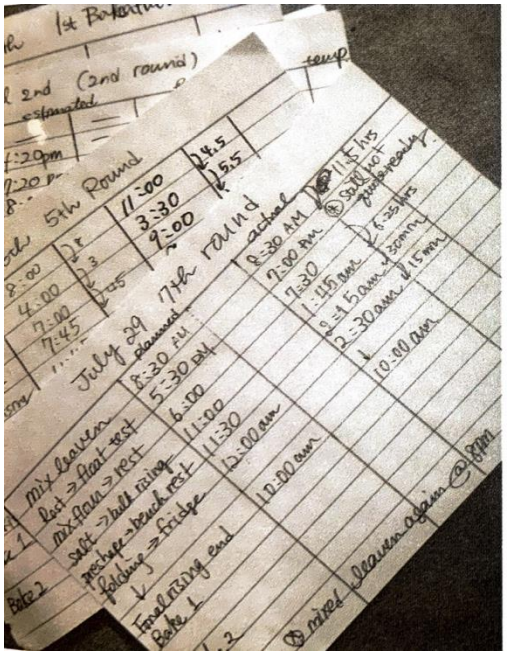
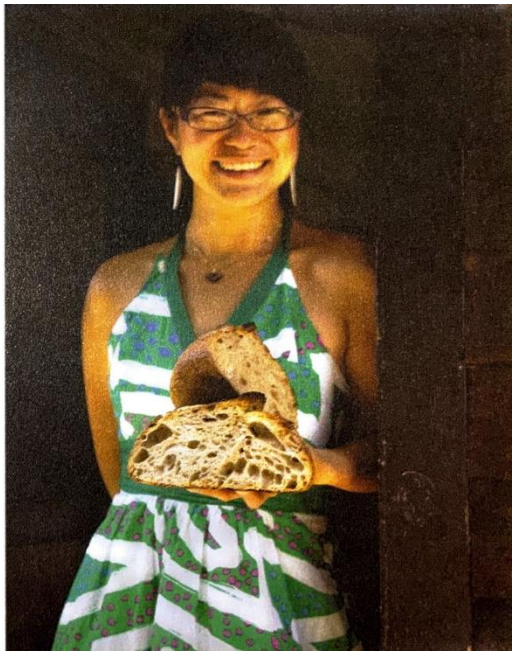
Version control





- dough rises slowly
- dough doesn't rise
- bread doesn't rise in oven
- taste: sour vs. not sour
- your favorite flour is out of stock

For coding:

- some file format changes, so you update your code
- you find a bug in your code, so you need to fix it
- Always have a continuous "before and after" record for other work – why not for your code as well?

Version control

You might implement some kind of versioning system already

PHYTOTAXA_lindavia

Name	Date Modified	Size	Kind
18_feb_2015	Feb 22, 2015, 3:50 PM	--	Folder
houk_klee_tanaka_scans	Jan 15, 2015, 12:35 PM	--	Folder
resubmission	May 13, 2015, 8:28 PM	--	Folder
reviews	May 10, 2015, 10:45 PM	--	Folder
submitted	May 10, 2015, 8:27 PM	--	Folder
lindavia_v1.docx	Dec 14, 2014, 8:15 PM	85 KB	Word
lindavia_v2_tn_added_some_discussion.AJA.docx	Jan 9, 2015, 9:57 AM	93 KB	Word
lindavia_v2_tn_added_some_discussion.docx	Jan 8, 2015, 1:25 PM	90 KB	Word
lindavia_v2.docx	Jan 6, 2015, 9:37 AM	87 KB	Word
lindavia_v3_outline_of_reduced_discussion.docx	Jan 10, 2015, 2:23 PM	93 KB	Word
lindavia_v4.docx	Jan 11, 2015, 11:54 AM	93 KB	Word
nakov_good_unused_discussion.docx	Feb 22, 2015, 2:09 PM	141 KB	Word
Schutt Interpretation v3.AJA.docx	Feb 7, 2015, 12:47 PM	19 KB	Word
Schutt Interpretation.AJA.docx	Feb 3, 2015, 9:30 PM	114 KB	Word
Schutt_Lindavia_C_socialis.docx	Feb 8, 2015, 2:13 PM	19 KB	Word
TN_Lindavia_draft_v1_2014-12-13.docx	Dec 14, 2014, 11:05 AM	52 KB	Word
0604Nakov_proof1.AJA.pdf	Jun 16, 2015, 10:35 AM	951 KB	PDF Document
Nakov_etal_2015.pdf	Oct 7, 2015, 11:04 AM	595 KB	PDF Document
Schutt.1899.C.socialis.drawings.text.Jahrbucher.pdf	Feb 5, 2015, 9:48 PM	3.3 MB	PDF Document

Things get particularly messy when you're collaborating.

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Version control

- A particularly vexing problem for software engineers
  - many different *version control systems*
  - Git is among the most popular
  - written by Linus Torvalds (the Linux dude)
  - designed to manage thousands of collaborators, all working simultaneously

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Version control with Git

1. Git allows you to keep snapshots of your project.
   - easily trace the origins of bugs and rollback changes
   - easily trace the evolution of your code
   - easily document your code – important for reproducibility
2. Git helps you keep track of important changes to code.
3. Git helps you keep software organized and available after people leave your lab.
4. Git allows you to share your code with others.
5. Git is widely used in the data science community.

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Install git

Configure Git

- Git commands take the form `'git <subcommand>'`
- Configure Git

```
$ git config --global user.name "Andrew Alverson"
$ git config --global user.email "aja@uark.edu"
$ git config --global color.ui true
$ git config --global core.editor nano
$ git config --list
```

Create a repository from scratch

Create a local repository in directory called `'bio_project'`

```
bio_project $ cd /Users/aja/Desktop/biol5153/
/Users/aja/Desktop/biol5153

biol5153 $ mkdir bio_project

biol5153 $ cd bio_project/
/Users/aja/Desktop/biol5153/bio_project

bio_project $ git init
Initialized empty Git repository in /Users/aja/Desktop/biol5153/bio_project/.git/

bio_project $ ls -al
total 0
drwxr-xr-x   3 aja  staff  102 Mar 21 10:07 .
drwxr-xr-x  14 aja  staff  476 Mar 21 10:06 ..
drwxr-xr-x  10 aja  staff  340 Mar 21 10:07 .git

bio_project $
```

how/where Git manages your repository

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Keeping track of files with Git

Add content to `'bio_project'` and check the status of the files

```
bio_project $ mkdir data
bio_project $ touch data/README data/file1.fasta data/file2.fasta
bio_project $ touch README
bio_project $ git status
On branch main
```

← we're on the *main* branch, the default branch for Git

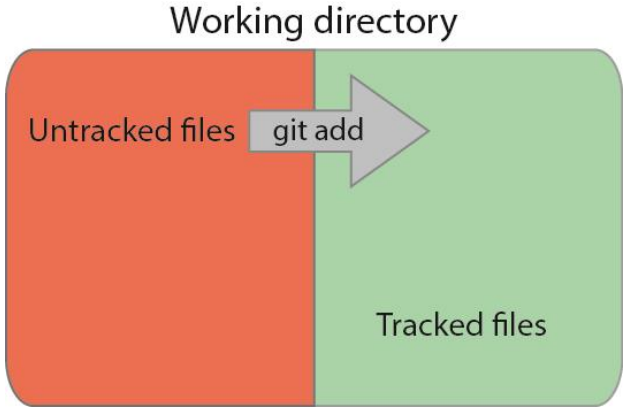
```
Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    README
    data/
```

← a list of files in the directory that Git is not tracking

```
nothing added to commit but untracked files present (use "git add" to track)

bio_project $
```

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Tracking files with `git add`



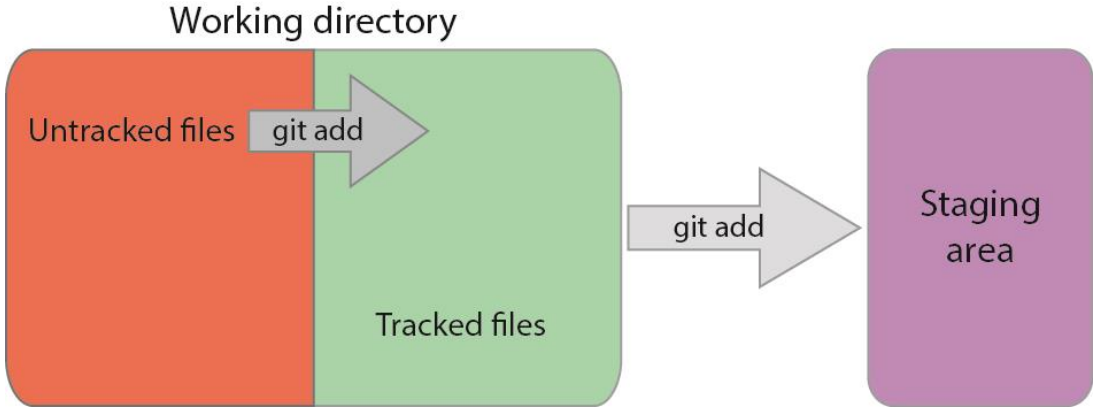
Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Tracking files with `git add`

```
bio_project $ git add README data/README
bio_project $ git status
On branch main

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   README
    new file:   data/README

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    data/file1.fasta
    data/file2.fasta

bio_project $
```

when first tracking a file, its current version is staged

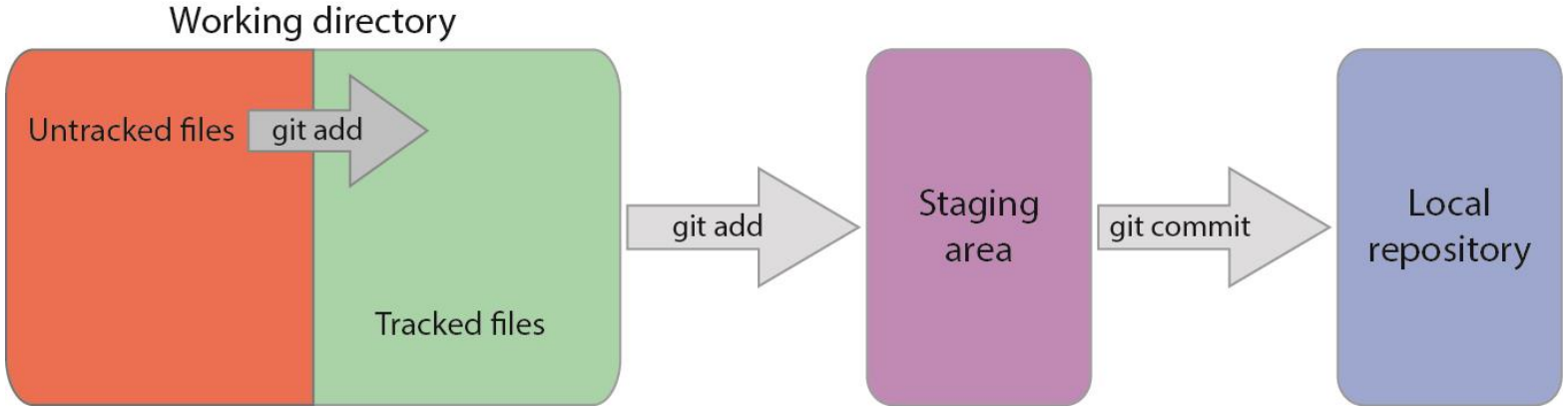
Staging files with `git add`



- a file that's **tracked** just means Git knows about it
- a file that's **staged** means it's **tracked** *and* the latest changes are ready to be archived, i.e., included in the next **commit**
- new changes are not automatically stored – we need to explicitly tell Git when to archive a set of changes with `git commit`

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

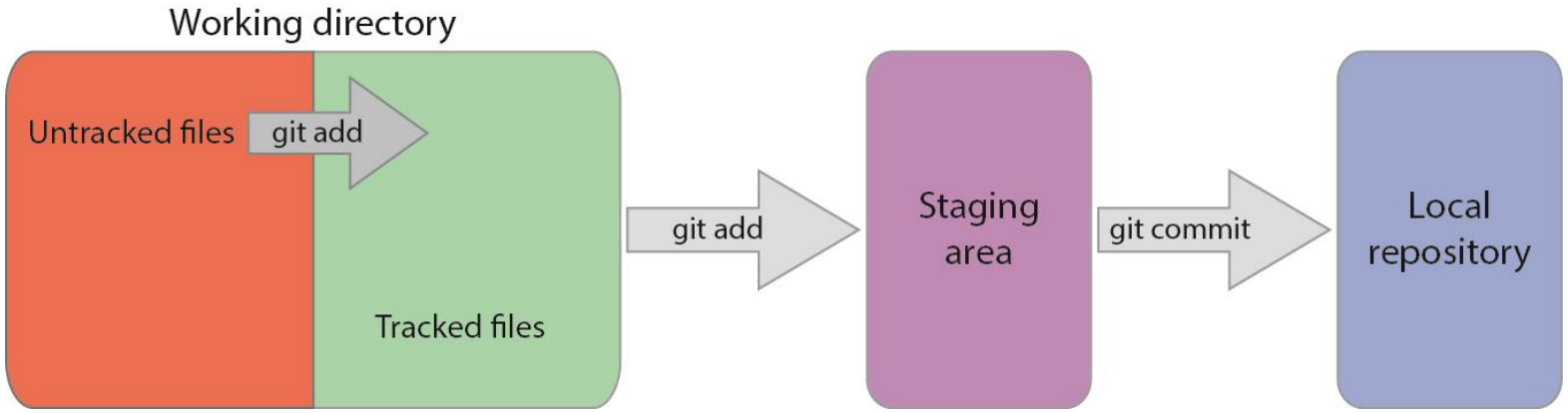
Storing changes with `git commit`



Why do I have to stage files first?

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

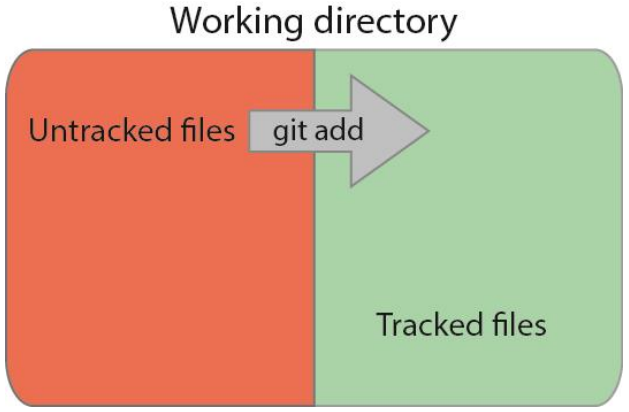
Staging is what you want.



- You'll often work on many different files in a project
- Changes will be complete for some files, incomplete for others
- You don't want to commit and store changes at a meaningless stage of development
- Staging allows you to commit changes for select files at meaningful points of development
- Alternative is to take snapshots of the entire set of tracked files *en masse* – those snapshots will be pretty random for some files

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Tracking files with `git add`



Remember, `git add` does two things:

1. alerts Git to start tracking a file
2. stages changes made to a file that's already being tracked

Important: Changes made to a file since the last time it was staged will not be included in the next commit unless they are staged with `git add`.

Staging files with `git add`

```
bio_project $ echo "The README file for my project" >> README
bio_project $ git status
On branch main

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   README
    new file:   data/README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   README

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    data/file1.fasta
    data/file2.fasta

bio_project $
```

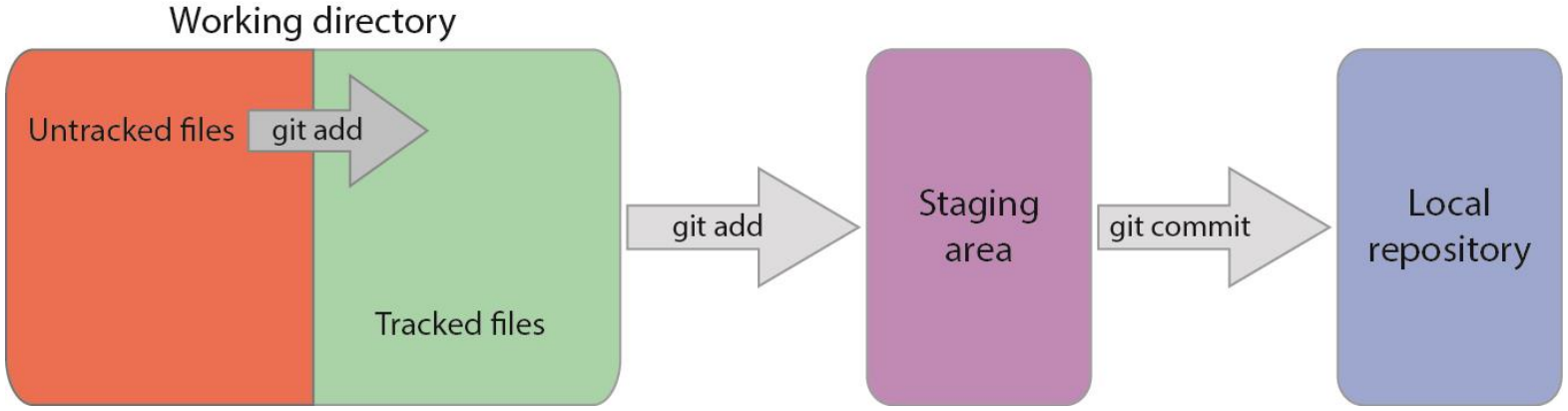
committing now would not include the change above

Staging files with `git add`

Stage the updated version of `README`

```
bio_project $ git add README
bio_project $ git status
On branch main

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   README
    new file:   data/README

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    data/file1.fasta
    data/file2.fasta

bio_project $
```

Storing changes with `git commit`



Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Storing your changes with `git commit`

Commit your changes

```
bio_project $ git commit -m "initial import"
[main (root-commit) 19ad95b] initial import
 2 files changed, 1 insertion(+)
 create mode 100644 README
 create mode 100644 data/README
bio_project $
```

Check the status

```
bio_project $ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    data/file1.fasta
    data/file2.fasta

nothing added to commit but untracked files present (use "git add" to track)
bio_project $
```

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Provide informative commit messages

http://xkcd.com/1296/

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Undoing a stage with `git reset`

If you change your mind about a file that's not ready for staging, you can unstage it with `git reset`

```
bio_project $ echo "Wrote the best script of my life" >> README
bio_project $ git add README
bio_project $ git status
On branch main
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   README.md

bio_project $
```

Follow Git's instructions (note: `HEAD` is a pointer to the last commit on our current branch)

```
bio_project $ git reset HEAD README
Unstaged changes after reset:
M   README.md
bio_project $
```

Ignore files with `.gitignore`

Tell Git to stop reminding us that our data files aren't being tracked

1. create a file called `.gitignore` in the `'bio_project'` directory
2. add a list of files (one per line) that you want Git to ignore, for example: `data/*.fasta`

```
bio_project $ echo 'data/*.fasta' > .gitignore
bio_project $ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    .gitignore

nothing added to commit but untracked files present (use "git add" to track)

bio_project $ git add .gitignore
bio_project $ git commit .gitignore -m "adding .gitignore"
[main 28d84f4] adding .gitignore
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore

bio_project $ git status
On branch main
nothing to commit, working directory clean
bio_project $
```

now we have a "clean" directory with no annoying messages

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Viewing your commit history with `git log`

Use `git log` to see all of your commits

```
bio_project $ git log --name-status
commit 6f274dbbcd32148a658e3b970e9090d042042bd4
Author: andrewalverson <andy.alverson@gmail.com>
Date:   Tue Mar 22 11:42:51 2016 +0300

    added project start date

M       README

commit 28d84f4de818b85c79ca400c5cd96e47e5644b07
Author: andrewalverson <andy.alverson@gmail.com>
Date:   Mon Mar 21 20:43:11 2016 +0300

    adding .gitignore

A       .gitignore

commit 19ad95bfa5e1a0fe356cf0e4fb53eafbde210f6b
Author: andrewalverson <andy.alverson@gmail.com>
Date:   Mon Mar 21 20:34:04 2016 +0300

    initial import

A       README
A       data/README
bio_project $ git log --abbrev-commit --pretty=oneline -n 3
```

SHA-1 checksum provides a unique ID for each commit

Renaming and deleting tracked files

Git will become confused if you delete or rename tracked files, so you have to do it Git's way

```
bio_project $ git mv README README.md
bio_project $ git mv data/README data/README.md
bio_project $ git status
On branch main
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README -> README.md
    renamed:    data/README -> data/README.md

bio_project $ git commit -m "add markdown extenstion to README files"
[main 989d893] add markdown extenstion to README files
 2 files changed, 0 insertions(+), 0 deletions(-)
 rename README => README.md (100%)
 rename data/{README => README.md} (100%)
bio_project $ git status
On branch main
nothing to commit, working directory clean
bio_project $
```

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Retrieving old versions with `checkout`

You've archived old versions, revert/restore with `git checkout`

```
bio_project $ git log README.md
commit 25ce9c2844274db2b9973fa42c7ccacd3dc6264e
Author: andrewalverson <andy.alverson@gmail.com>
Date:   Wed Mar 30 12:45:13 2016 -0500

    added data to README

commit 989d893a7342a57a225c28895b278eaa436d4553
Author: andrewalverson <andy.alverson@gmail.com>
Date:   Tue Mar 22 12:04:25 2016 +0300

    add markdown extenstion to README files
bio_project $ git checkout 989d893a7342a57a225c28895b278eaa436d4553 README.md
bio_project $ cat README.md
The README file for my project
Project started 2016-02-14
bio_project $
```

Collaborating with Git

Version control

You probably implement some kind of versioning system already

PHYTOTAXA_lindavia

Name	Date Modified	Size	Kind
18_feb_2015	Feb 22, 2015, 3:50 PM	--	Folder
houk_klee_tanaka_scans	Jan 15, 2015, 12:35 PM	--	Folder
resubmission	May 13, 2015, 8:28 PM	--	Folder
reviews	May 10, 2015, 10:45 PM	--	Folder
submitted	May 10, 2015, 8:27 PM	--	Folder
lindavia_v1.docx	Dec 14, 2014, 8:15 PM	85 KB	Word
lindavia_v2_tn_added_some_discussion.AJA.docx	Jan 9, 2015, 9:57 AM	93 KB	Word
lindavia_v2_tn_added_some_discussion.docx	Jan 8, 2015, 1:25 PM	90 KB	Word
lindavia_v2.docx	Jan 6, 2015, 9:37 AM	87 KB	Word
lindavia_v3_outline_of_reduced_discussion.docx	Jan 10, 2015, 2:23 PM	93 KB	Word
lindavia_v4.docx	Jan 11, 2015, 11:54 AM	93 KB	Word
nakov_good_unused_discussion.docx	Feb 22, 2015, 2:09 PM	141 KB	Word
Schutt Interpretation v3.AJA.docx	Feb 7, 2015, 12:47 PM	19 KB	Word
Schutt Interpretation.AJA.docx	Feb 3, 2015, 9:30 PM	114 KB	Word
Schutt_Lindavia_C_socialis.docx	Feb 8, 2015, 2:13 PM	19 KB	Word
TN_Lindavia_draft_v1_2014-12-13.docx	Dec 14, 2014, 11:05 AM	52 KB	Word
0604Nakov_proof1.AJA.pdf	Jun 16, 2015, 10:35 AM	951 KB	PDF Document
Nakov_etal_2015.pdf	Oct 7, 2015, 11:04 AM	595 KB	PDF Document
Schutt.1899.C.socialis.drawings.text.Jahrbucher.pdf	Feb 5, 2015, 9:48 PM	3.3 MB	PDF Document

Things get particularly messy when you're collaborating.

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Clone an existing repository

- can clone a repository from anywhere
- most common to clone from a repository hosting service (e.g., GitHub)

```
biol5153 $ git clone https://github.com/lh3/seqtk.git
Cloning into 'seqtk'...
remote: Counting objects: 265, done.
remote: Total 265 (delta 0), reused 0 (delta 0), pack-reused 265
Receiving objects: 100% (265/265), 128.50 KiB | 5.00 KiB/s, done.
Resolving deltas: 100% (149/149), done.
Checking connectivity... done.

biol5153 $
```

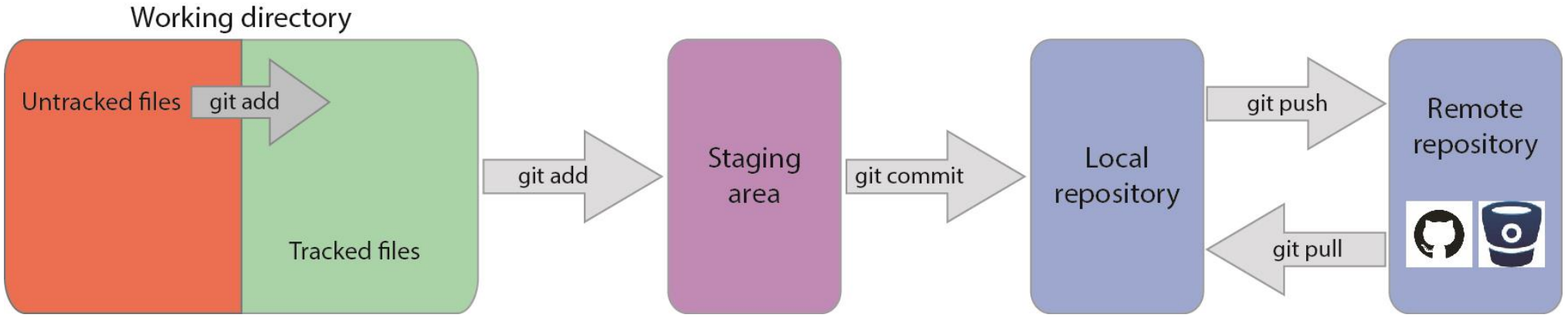
Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Clone an existing repository

You've cloned the seqtk repository.

```
biol5153 $ cd seqtk/
/Users/aja/Desktop/biol5153/seqtk
seqtk $ ls -al
total 304
drwxr-xr-x  16 aja  staff    544 Mar 21 10:13 .
drwxr-xr-x  15 aja  staff    510 Mar 21 10:12 ..
drwxr-xr-x  13 aja  staff    442 Mar 21 10:13 .git
-rw-r--r--   1 aja  staff     22 Mar 21 10:13 .gitignore
-rw-r--r--   1 aja  staff   1086 Mar 21 10:13 LICENSE
-rw-r--r--   1 aja  staff    303 Mar 21 10:13 Makefile
-rw-r--r--   1 aja  staff   1567 Mar 21 10:13 README.md
-rw-r--r--   1 aja  staff  18784 Mar 21 10:13 khash.h
-rw-r--r--   1 aja  staff   8795 Mar 21 10:13 kseq.h
-rw-r--r--   1 aja  staff  10524 Mar 21 10:13 ksort.h
-rw-r--r--   1 aja  staff   4439 Mar 21 10:13 kstring.h
-rw-r--r--   1 aja  staff  15865 Mar 21 10:13 ksw.c
-rw-r--r--   1 aja  staff   2396 Mar 21 10:13 ksw.h
-rw-r--r--   1 aja  staff   2880 Mar 21 10:13 kvec.h
-rw-r--r--   1 aja  staff  50583 Mar 21 10:13 seqtk.c
-rw-r--r--   1 aja  staff   5722 Mar 21 10:13 trimadap.c
seqtk $
```

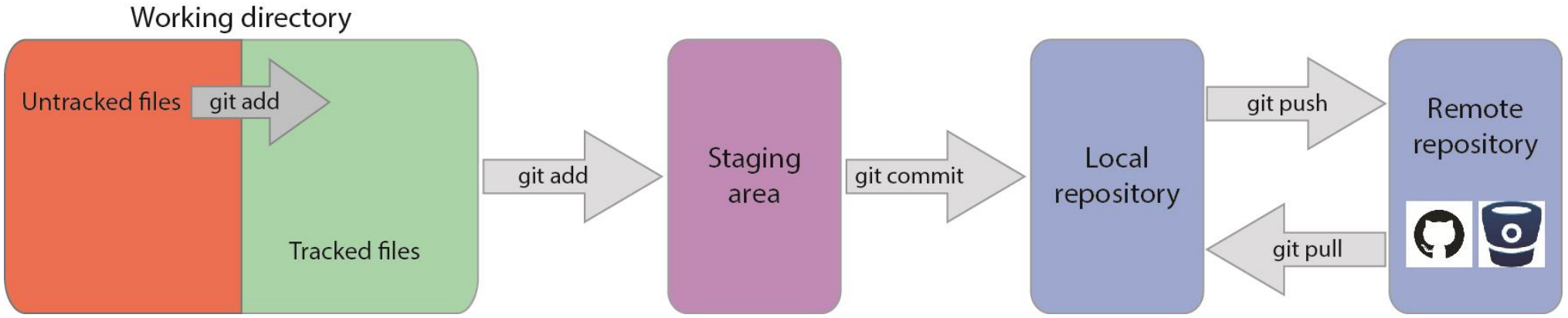
Version control with Git



Remote repository: a copy of your repository hosted elsewhere

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

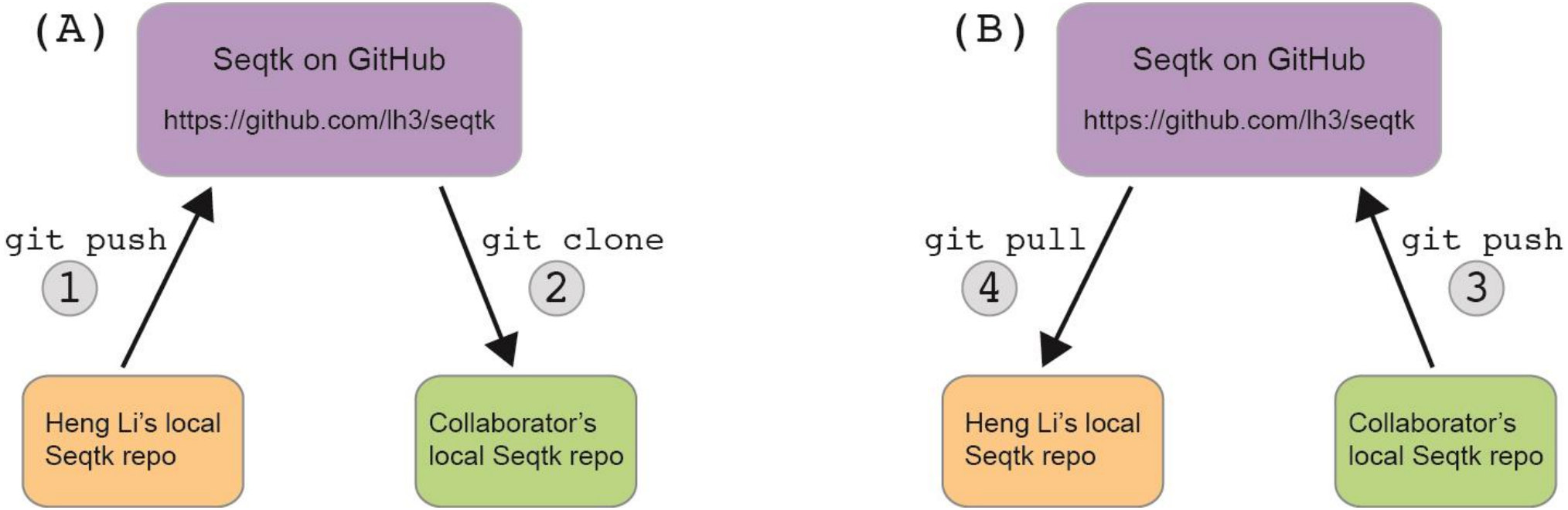
Version control with Git



- **`push`**: send a change to a shared central repository
- **`pull`**: grab a change from a repository

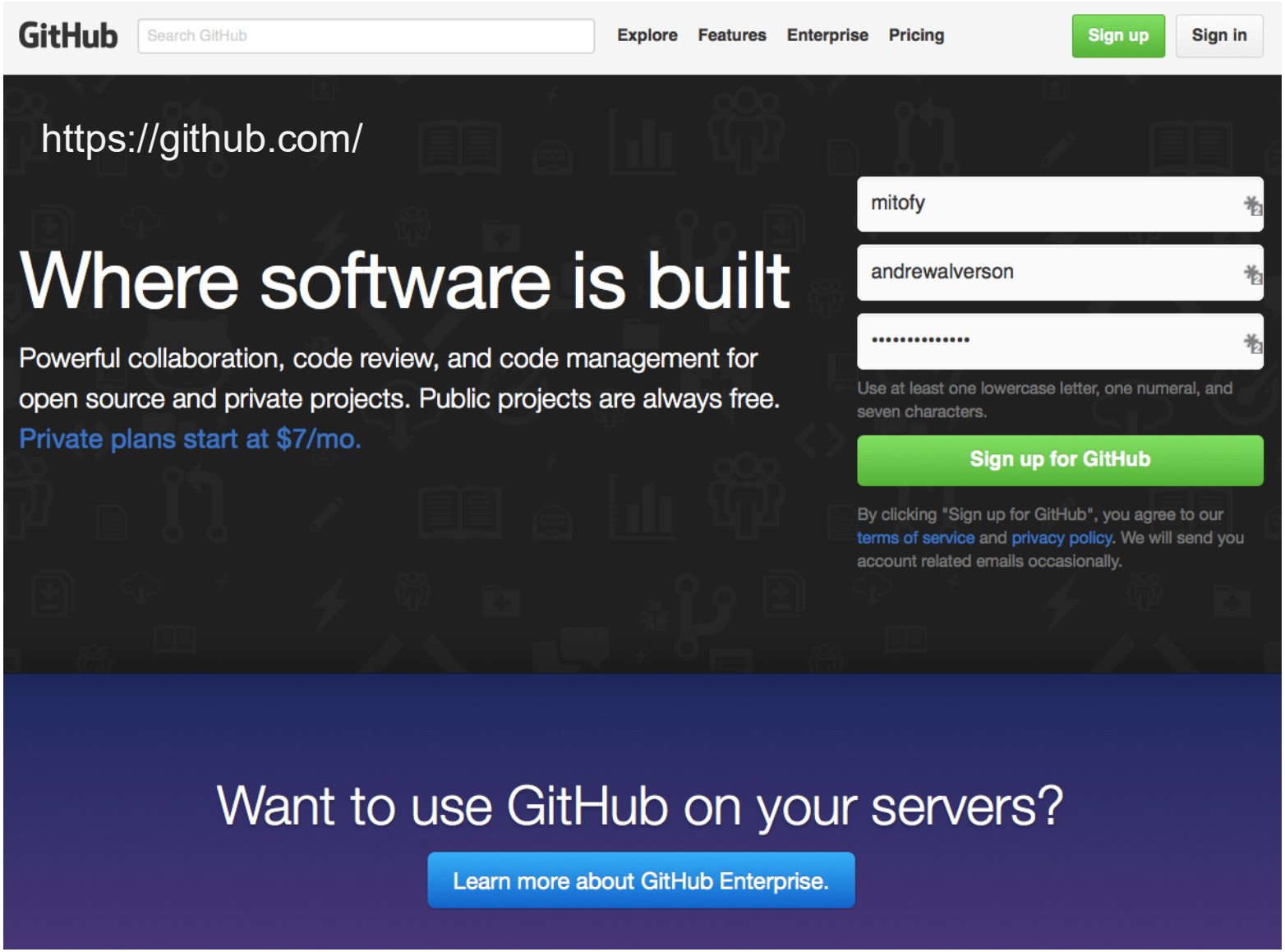
Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Collaborating through a shared central repository

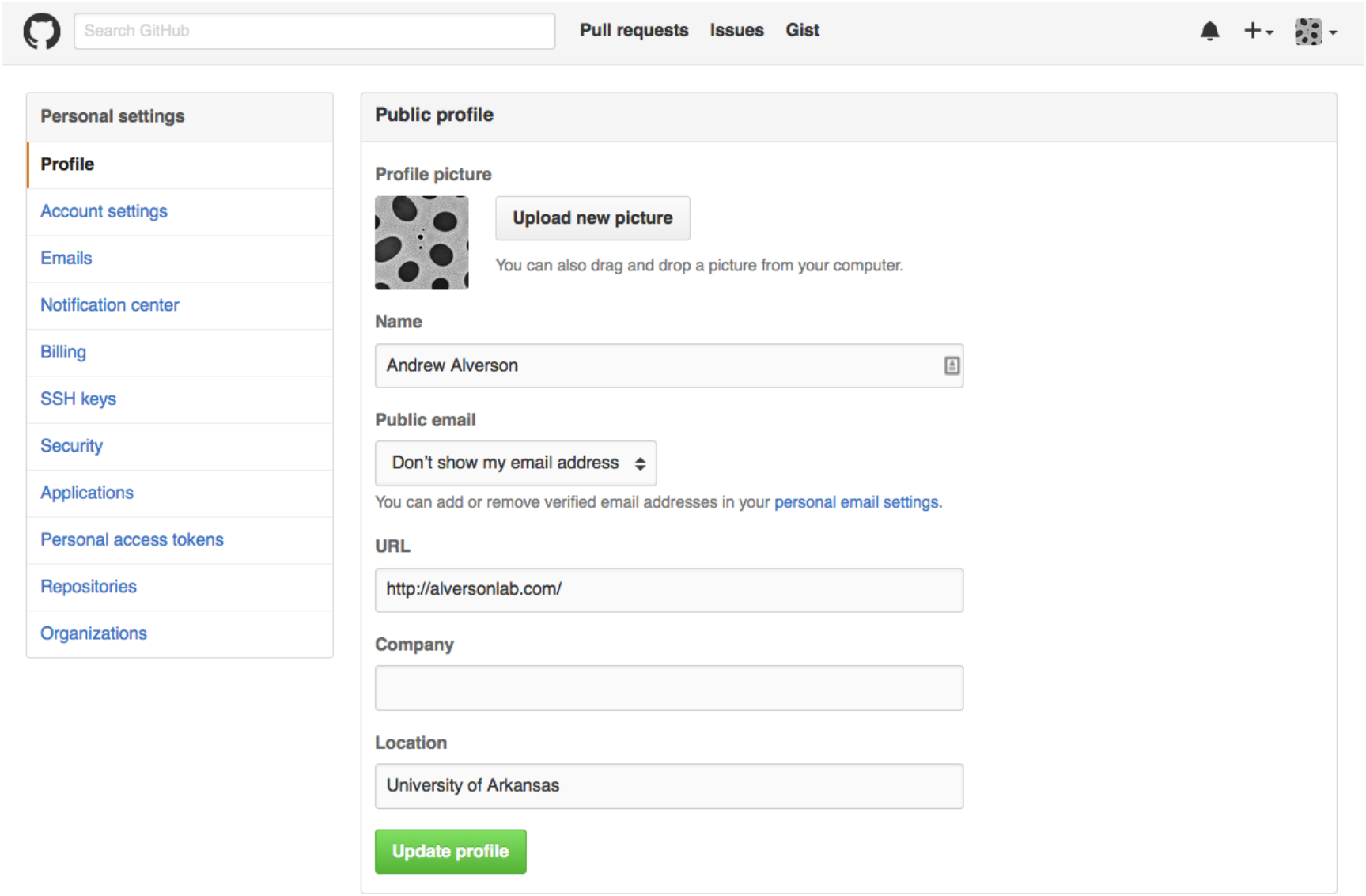


The commit log will be a mix of changes made by Heng Li and other contributors

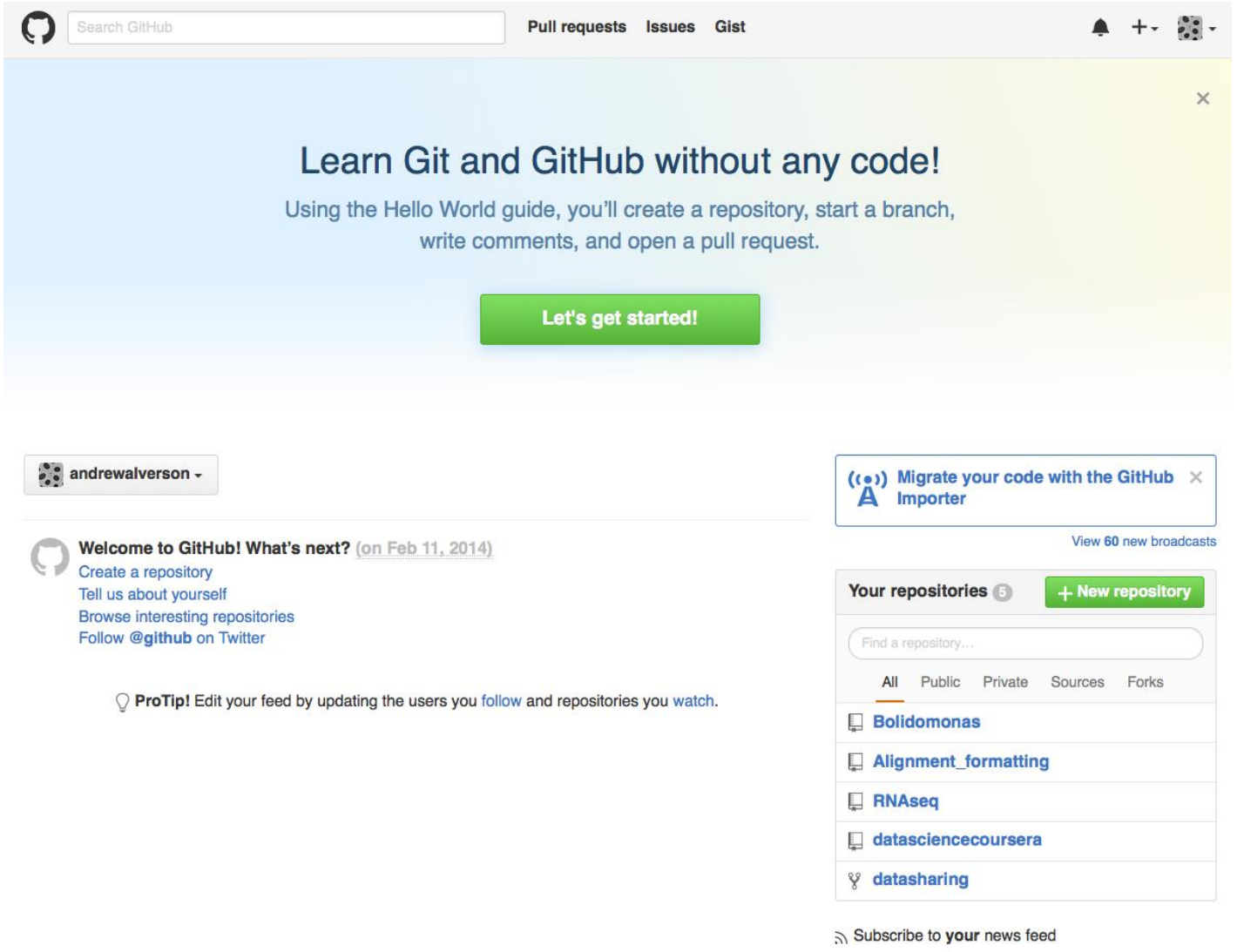
Create an account on Github



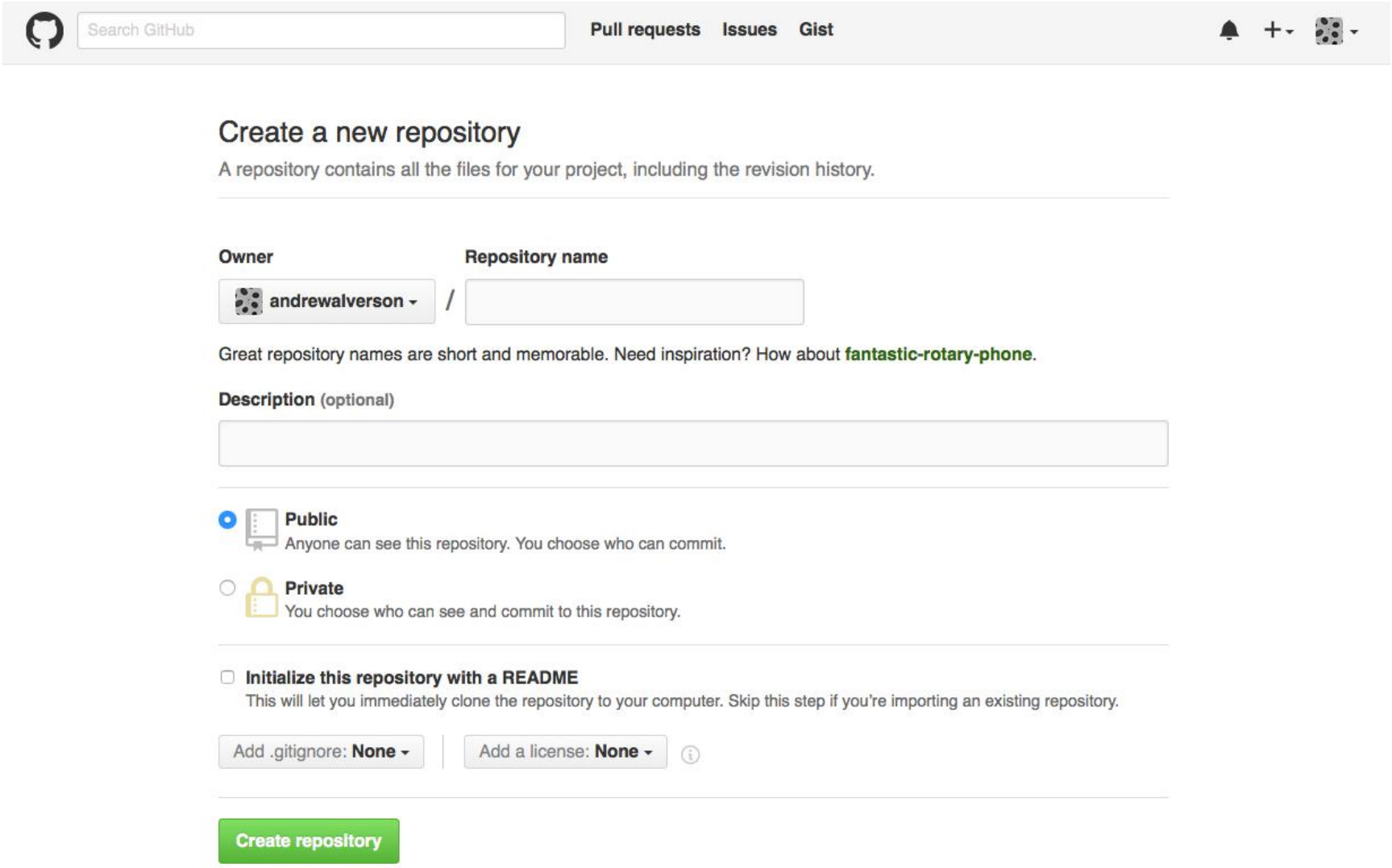
Explore your account settings



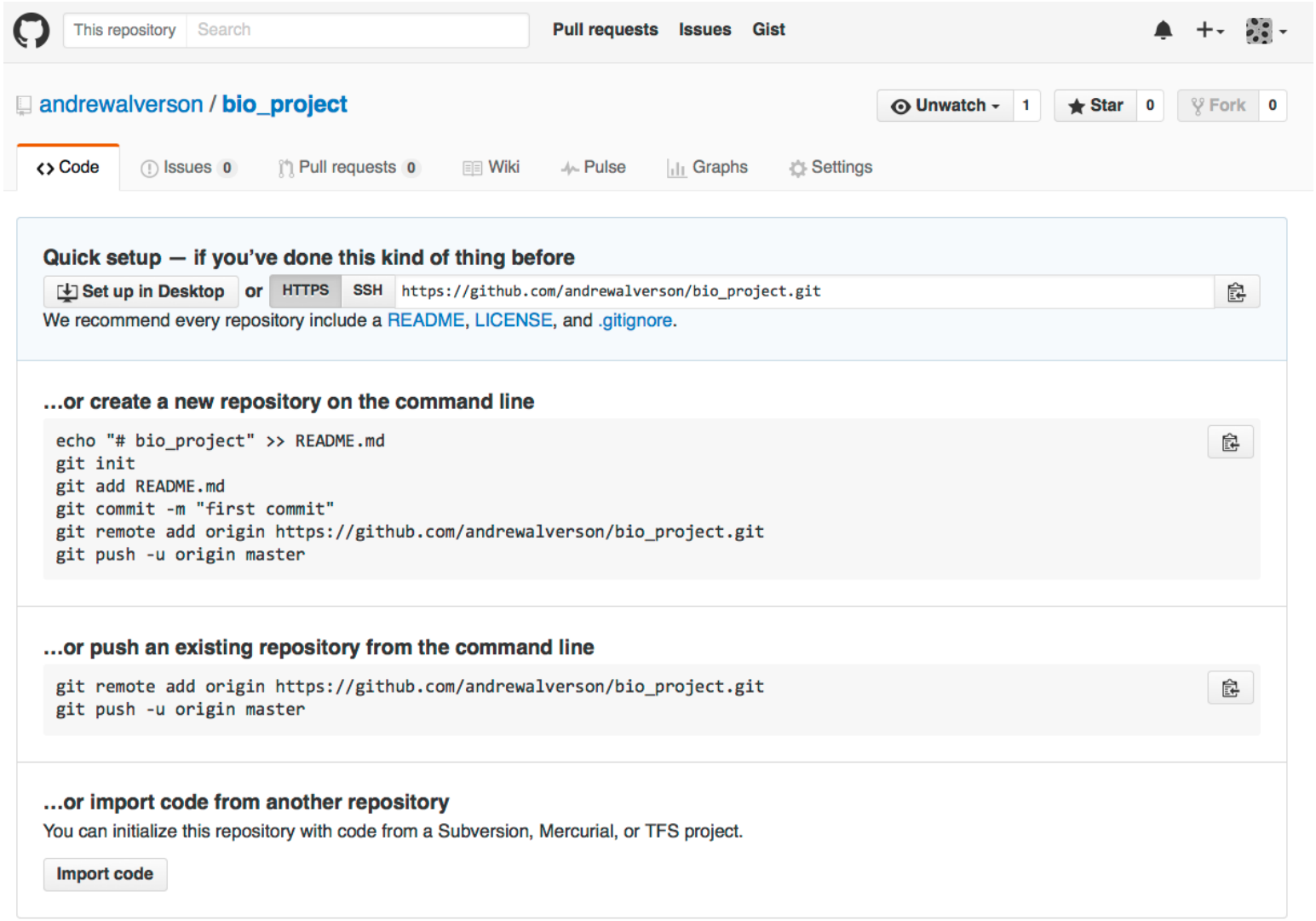
Create a new repository



Create a new repository



Create a new repository



Connecting with `git remote`

```
bio_project $ git remote add origin https://github.com/andrewalverson/bio_project.git
bio_project $ git push -u origin main
Counting objects: 17, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (12/12), done.
Writing objects: 100% (17/17), 1.57 KiB | 0 bytes/s, done.
Total 17 (delta 2), reused 0 (delta 0)
To https://github.com/andrewalverson/bio_project.git
 * [new branch]      main -> main
Branch main set up to track remote branch main from origin.
bio_project $
```

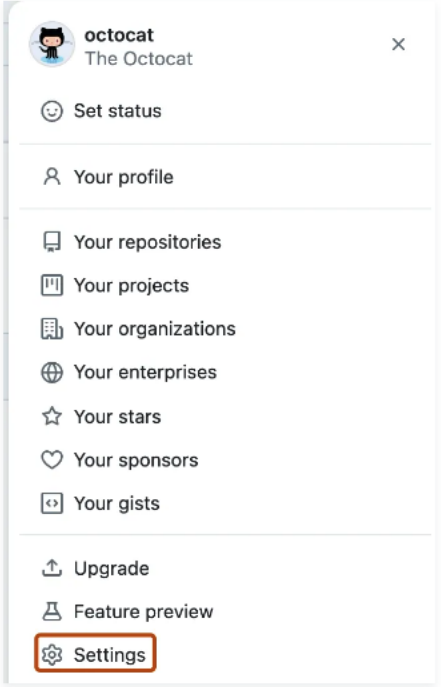
Authentication with a personal access token (Link)

- Select classic token
- Select all the permissions
- This will be your password

Creating a fine-grained personal access token

> **Note**: Fine-grained personal access token are currently in beta and subject to change. To leave feedback, see the feedback discussion.

1. Verify your email address, if it hasn't been verified yet.
2. In the upper-right corner of any page, click your profile photo, then click **Settings**.



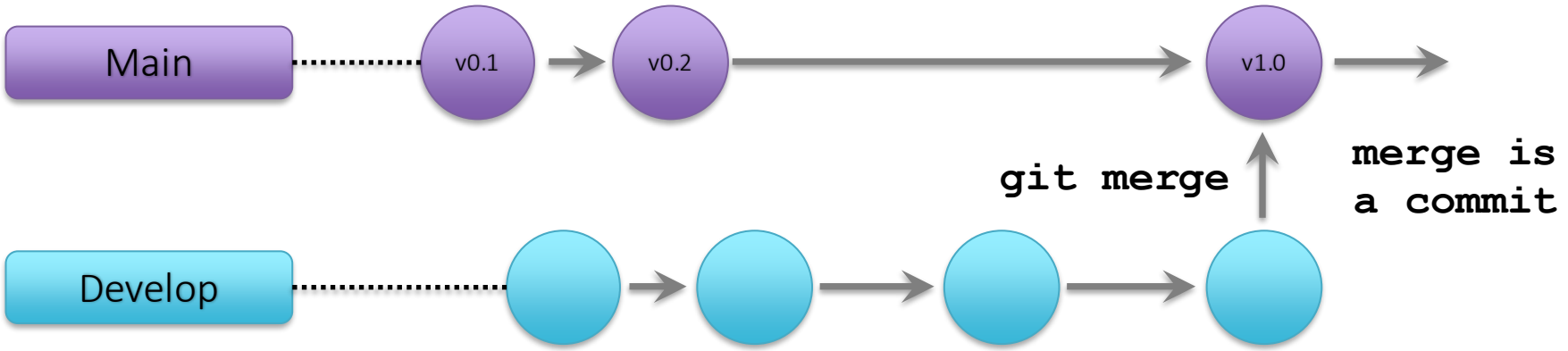
3. In the left sidebar, click <> **Developer settings**.
4. In the left sidebar, under **Personal access tokens**, click **Fine-grained tokens**.
5. Click **Generate new token**.
6. Under **Token name**, enter a name for the token.
7. Under **Expiration**, select an expiration for the token.
8. Optionally, under **Description**, add a note to describe the purpose of the token.

Working with branches

- Experiment with your project without risk of messing up the main, working main branch
- Add major new feature without affecting the current working version
  - you may find that adding the new feature required more work than anticipated
  - abandon the effort and revert to unscathed working version
- Once the major change is complete, merge with the main, incorporating it into the working production version
- Collaborators can work on their own separate branches and individually merge them onto the main
- It's all done virtually, i.e., without copying everything

Vince Buffalo (2015). Bioinformatics Data Skills. O'Reilly Media

Working with branches



Working with branches

Create, then switch to, a new branch called *readme-changes*

```
bio_project $ git branch readme-changes
bio_project $ git branch
* main
  readme-changes
bio_project $ git checkout readme-changes
M   README.md
Switched to branch 'readme-changes'
bio_project $ git branch
  main
* readme-changes
bio_project $
```

Now edit README.md on the *readme-changes* branch

Working with branches

Make and commit edits on the *readme-changes* branch

```
bio_project $ nano README.md
bio_project $ git add README.md
bio_project $ git commit -m "added data to README"
[readme-changes 25ce9c2] added data to README
 1 file changed, 4 insertions(+)
bio_project $
bio_project $ git log --abbrev-commit --pretty=oneline -n 3
25ce9c2 added data to README
989d893 add markdown extenstion to README files
6f274db added project start date
bio_project $
```

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Working with branches

Merge edits on the *readme-changes* branch to the *main* branch

1. switch to the branch you want to merge onto
2. merge changes

```
bio_project $ git branch
  main
* readme-changes
bio_project $ git checkout main
Switched to branch 'main'
bio_project $ git merge readme-changes
Updating 989d893..25ce9c2
Fast-forward
 README.md | 4 ++++
 1 file changed, 4 insertions(+)
bio_project $ cat README.md
```

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Extras

Compare files with `git diff`

Use `git diff` to view file changes

- show difference between file in your working directory to what's been staged
- if nothing's staged, show difference between last commit and current version

```
bio_project $ echo "Project started 2016-02-14" >> README
bio_project $ git diff
diff --git a/README b/README
index 551a348..7992ba1 100644
--- a/README
+++ b/README
@@ -1 +1,2 @@
 The README file for my project
+Project started 2016-02-14
bio_project $
```

1
2

1. Two file versions (a and b): '---' = original version, '+++' = modified version
2. The actual change
   - leading space: nothing changed
   - leading '+': added line
   - leading '-': deleted line
   - changed lines are shown as deleted, then added

Compare files with `git diff`

What's the result? Why?

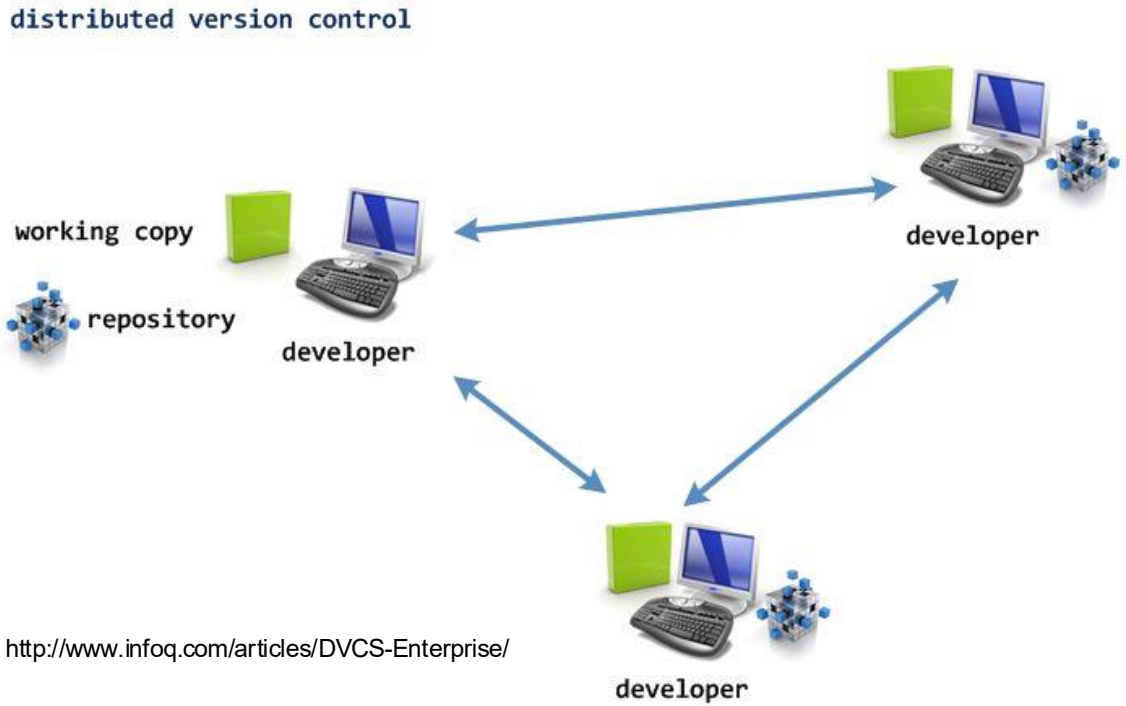
```
bio_project $ git add README
bio_project $ git diff
```

```
bio_project $ git diff --staged
diff --git a/README b/README
index 551a348..7992ba1 100644
--- a/README
+++ b/README
@@ -1 +1,2 @@
 The README file for my project
+Project started 2016-02-14
bio_project $
```

- By default, `git diff` shows the difference between files in your wd and what's been staged.
- If changes haven't been staged, `git diff` shows the difference between your last commit and the current version

Vince Buffalo (2015). *Bioinformatics Data Skills.* O'Reilly Media

Distributed version control

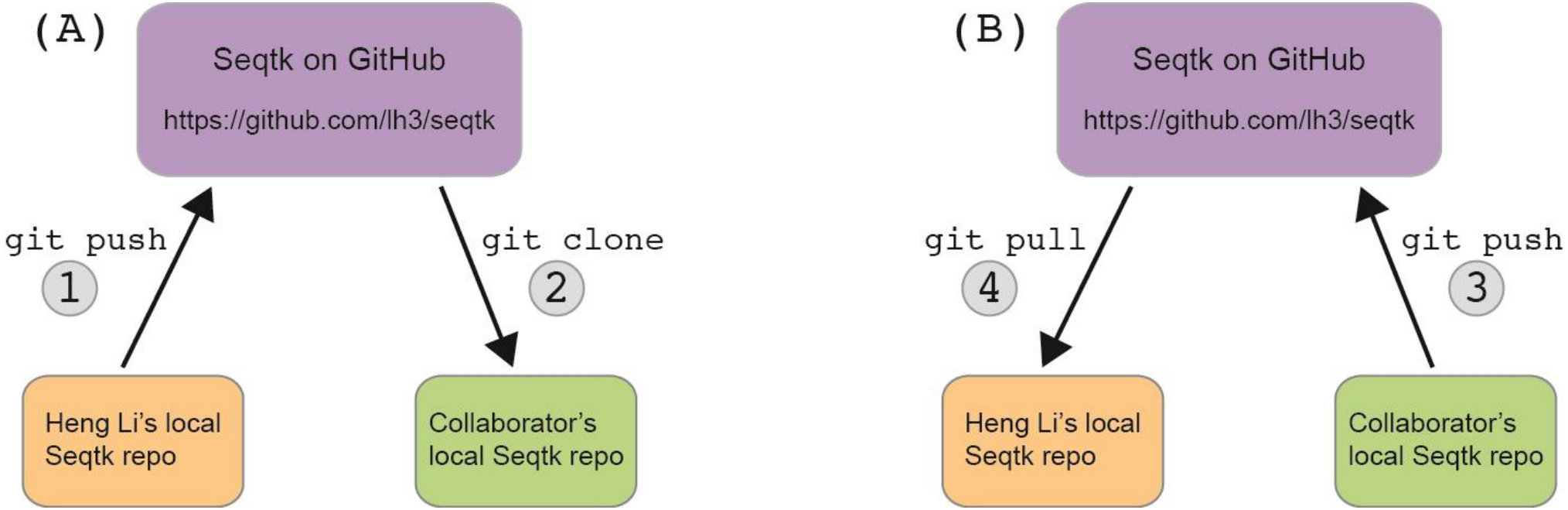


http://www.infoq.com/articles/DVCS-Enterprise/

In a **distributed** model, every developer has her own repo.

- Jill's changes live in **her local repo**, Betsy's changes live in **her local repo**, etc.
- Each person can share her changes with all the other collaborators
- Compare with Centralized Version Control

Collaborating through a shared central repository



- Merge conflicts can arise when two people are working on the same file
- These need to be manually resolved